CAREER ROADMAP SERIES

ZERO TO HIRED: AI ENGINEERING ROADMAP (2026)

YOUR AI CAREER STARTS HERE



> From absolute beginner to landing your first **$100k+ AI role.**
>
> Master modern LLM architectures, **RAG, Agents & deployment** without a PhD.

6-STAGE ROADMAP

1. PYTHON & APIS
2. EMBEDDINGS
3. LLM SDKS
4. PRODUCTION RAG
5. AI AGENTS
6. EVALUATION & DEPLOY

6 MONTHS STRUCTURED PLAN

5 REAL PROJECTS RESUME READY

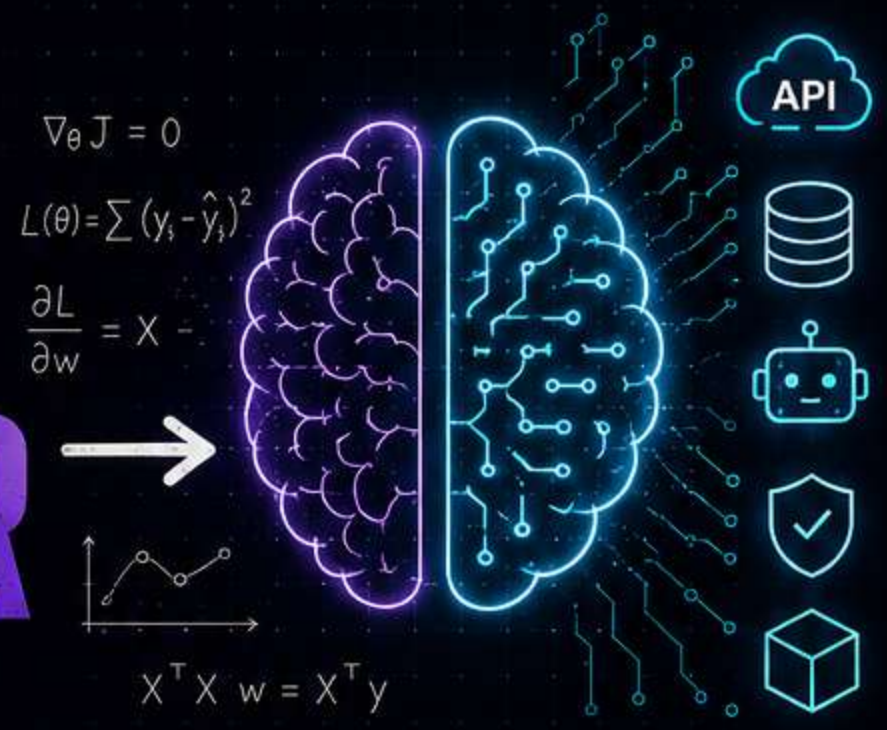
ZERO MATH FLUFF PRODUCTION FOCUS

SWIPE TO START YOUR AI ENGINEERING ROADMAP →

resnur.ai | By Samarasimha Reddy Peyala

The Mindset Shift:

WHAT IS AN AI ENGINEER IN 2026?



OLD PATH



ML RESEARCHER / DATA SCIENTIST

- Spends months training models from scratch
- Writing complex PyTorch math proofs
- Tuning gradient loss & chasing accuracy



Vs

MODERN HIGH-DEMAND PATH



AI ENGINEER

- Builds production software **around** state-of-the-art foundation models
- Uses APIs, RAG, Agents, Guardrails & Vector DBs
- Ships solutions that create real-world impact



CORE TAKEAWAYS

You do **NOT** need a PhD in math or 5 years of linear algebra.

You **DO** need solid software engineering, API integration, and system design skills.

OVER 85% of modern AI job openings in 2026 require **AI Engineering & RAG** skills rather than custom model training.

Month 1: MODERN PYTHON & PRODUCTION TOOLING



Build the rock-solid foundation for your AI Engineering journey.

Python 3.12+ → Type Hinting → Pydantic v2 → FastAPI → Docker

WHAT TO MASTER

Python 3.11/3.12+:

Type hinting, list comprehensions, decorators, context managers (with).

BEST RESOURCES

- Bro Code – Python Full Course (2024)
- JetBrains – Python Type Hinting Guide
- Real Python – Advanced Python 3

Pydantic v2:

Data validation and JSON schema parsing (crucial for LLM structured outputs).

BEST RESOURCES

- Coding with Mosh – Pydantic v2 Tutorial
- Pydantic Official Docs
- Udemy – FastAPI & Pydantic v2

Asyncio & Await:

Concurrency for high-throughput LLM API calls without blocking.

BEST RESOURCES

- Very Academy – Asyncio in Python
- Official Python Docs – asyncio
- Udemy – Python Asyncio Masterclass

FastAPI:

Building REST endpoints with streaming Server-Sent Events (SSE).

BEST RESOURCES

- Tech With Tim – FastAPI Full Course
- FastAPI Official Docs
- Udemy – FastAPI: Build APIs with Python

Docker:

Containerize your API for consistent production deployments.

BEST RESOURCES

- TechWorld with Nana – Docker Tutorial
- Docker Official Docs
- freeCodeCamp – Docker Course

WHAT TO SKIP

- Complex C++ bindings
- Low-level socket programming

HIRING INSIGHT

Hiring managers reject resumes with messy notebook scripts.

Clean, typed, modular code gets interviews.



Project 1:

PRODUCTION-READY STREAMING MICROSERVICE



Build it. Containerize it. Ship it.

CLIENT REQUEST
User sends a prompt

→

FASTAPI ASYNC HANDLER
Handles request asynchronously

→

PYDANTIC VALIDATION
Validates request & structures response

→

SSE STREAM CHUNK
Streams tokens in real-time

→

DOCKER CONTAINER
Packaged & ready to deploy

PROJECT SPECIFICATIONS

Build a REST API with **FastAPI** that takes a prompt and streams tokens back to the client in real-time using SSE.

Implement **Pydantic v2** request/response validation with meaningful error handling.

Containerize the entire application inside a **Dockerfile** for consistent deployment.

WHAT YOU WILL BUILD

- POST /stream endpoint
- Accepts JSON: {"prompt": "...."}
- Streams tokens as they are generated
- Proper error handling & status codes
- Dockerized & ready to run anywhere

TOOLS & TECHNOLOGIES

FastAPI · Pydantic v2 · SSE (Server-Sent Events) · Docker

LEARN & BUILD WITH THESE RESOURCES

DOCUMENTATION

- FastAPI Official Docs
- Pydantic v2 Documentation
- Docker Docs
- MDN Web Docs – SSE Guide

YOUTUBE PLAYLISTS

- Tech With Tim – FastAPI Full Course
- Coding with Mosh – FastAPI Tutorial
- Fireship – Server Sent Events (SSE)
- TechWorld with Nana – Docker Tutorial

EXAMPLE PROJECTS

- `openai/openai-cookbook (streaming examples)`
- `tiangolo/full-stack-fastapi-template (production ready FastAPI project)`

RESUME BULLET POINT YOU GET

> "Architected a containerized FastAPI microservice with async SSE streaming and strict Pydantic validation, deployed via Docker."

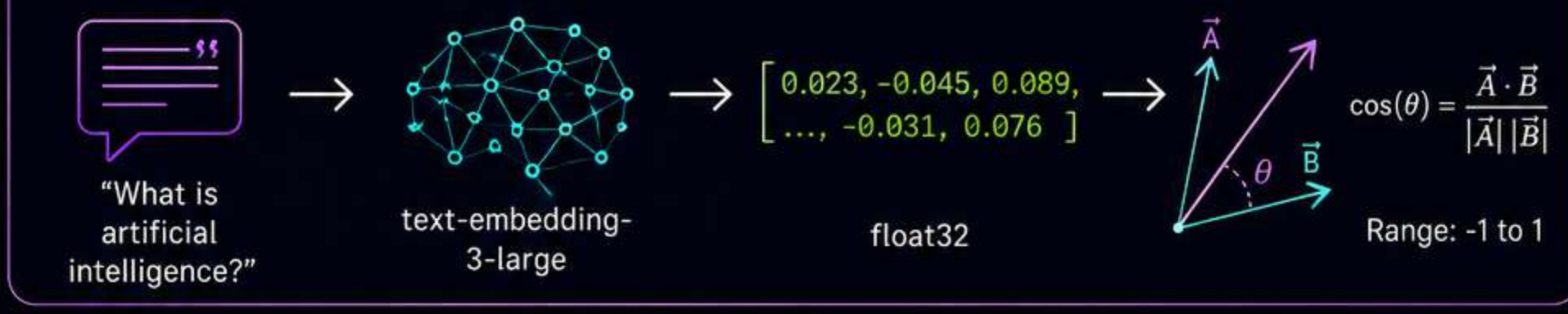
EXTRA CRED (IF YOU GO BEYOND)

- Add rate limiting (SlowAPI)
- Add logging (structlog)
- Health check endpoint (/health)
- Environment config with .env
- Deploy on Render / Railway / Fly.io

Month 2:

DATA HANDLING & VECTOR SPACE MECHANICS

Master data, embeddings, and the math behind intelligent retrieval.



WHAT TO MASTER

SQL & PostgreSQL:

Relational data querying, joins, indexing, and JSONB storage.

```sql
SELECT id, metadata->>'title' as title
FROM documents
WHERE metadata @> '{"type": "article"}'
ORDER BY created_at DESC
LIMIT 10;
```



Pandas & NumPy:

Data cleaning, vector math, inner dot products.

```python
import numpy as np
A = np.array([0.1, 0.2, 0.3])
B = np.array([0.4, 0.5, 0.6])
dot = np.dot(A, B)  # inner dot product
```

$$A = \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \end{bmatrix} \quad B = \begin{bmatrix} 0.4 \\ 0.5 \\ 0.6 \end{bmatrix} = 0.32$$

Dense Embeddings:

How text converts into float32 vector coordinates.

Text → Vector (1536 dimensions)

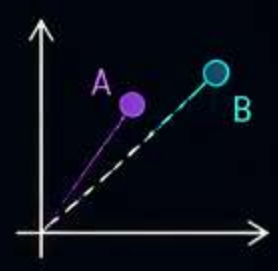
"AI changes the world." → [0.012, -0.008, ..., 0.031]

1536 float32 values

Vector Similarity Metrics:

Cosine Similarity, Dot Product, and Euclidean Distance.

- Cosine Similarity: $\cos(\theta) = \frac{\vec{A} \cdot \vec{B}}{|\vec{A}|\,|\vec{B}|}$
- Dot Product: $A \cdot B = \sum_{i=1}^{n} A_i B_i$
- Euclidean Distance: $\|A - B\|_2 = \sqrt{\sum_{i=1}^{n} (A_i - B_i)^2}$



WHAT TO SKIP

Deriving Backpropagation from scratch by hand.

Focus on tools and systems that ship real-world value.

FACT

Understanding Cosine Similarity is

90%

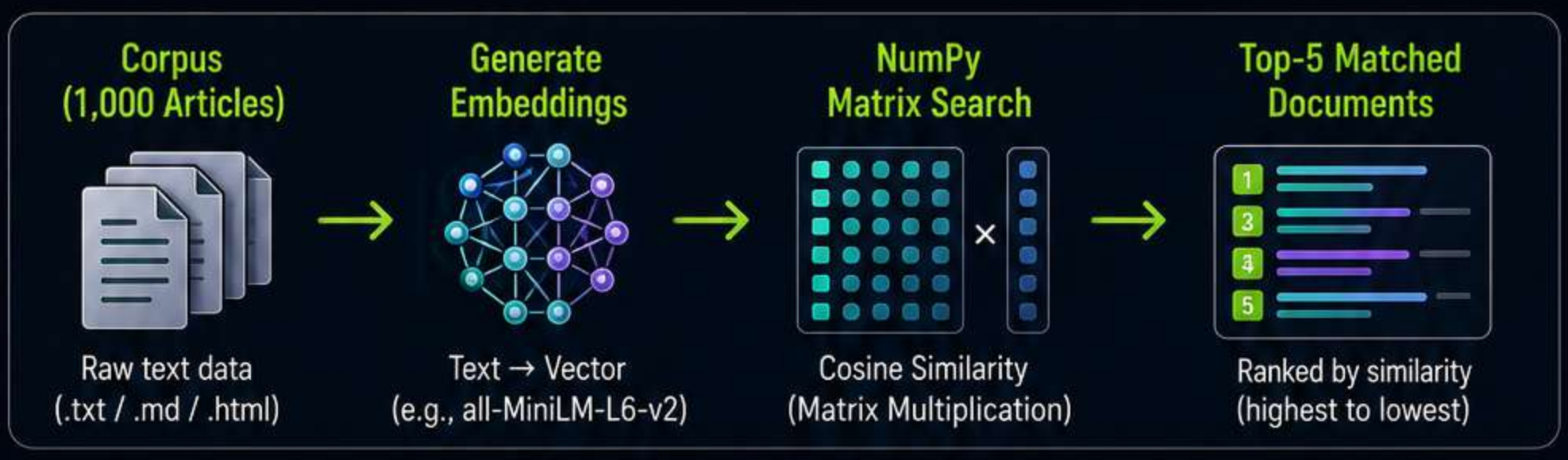
of the math needed for building production semantic search.



Project 2: Semantic Search Engine From Scratch



Build it raw. Understand it deep. Scale it right.



PROJECT SPECIFICATIONS

01 Build From First Principles

Build a semantic search engine using **raw NumPy matrix multiplication** before touching any vector database.

Key Concepts

- Text Preprocessing
- Embedding Generation
- Cosine Similarity (Manual)
- Efficient Matrix Operations

02 Benchmark Performance

10,000
Text Chunks

Benchmark **Cosine Similarity** speed across **10,000** text chunks and analyze memory + latency trade-offs.

Metrics to Track

- Throughput (queries/sec)
- Average Latency (ms)
- Memory Usage (MB)
- Scalability Limit

03 Migrate to pgvector

pgvector

Migrate the vector index to **pgvector** inside PostgreSQL for real-world scale and production readiness.

Focus Areas

- Create pgvector extension
- Index vectors in PostgreSQL
- Run similarity search with SQL
- Compare: NumPy vs pgvector

RESUME BULLET POINT

"Built high-performance vector search engine indexing 10k+ records using **pgvector** and cosine distance metrics."

PRO TIP

Don't start with Pinecone or MongoDB Atlas.
Understand the math → then use the tools.

Month 3:

FOUNDATION MODEL SDKS & STRUCTURED OUTPUTS



Use the best foundation models via APIs and get reliable, structured data out—every single time.

USER PROMPT

"Extract patient info from this note."

→

OPENAI / ANTHROPIC SDK

Using official SDKs (Streaming / Async)

→

LOGIT GRAMMAR MASKING

Constrain output tokens to follow a valid schema

→

VALID PYDANTIC JSON

```
{
  "name": "John Doe",
  "age": 42,
  "diagnosis": "Flu"
}
```

100% reliable structured JSON output

WHAT TO MASTER

Official SDKs

Work with OpenAI, Anthropic Claude, and Google Gemini APIs.

OpenAI
Claude
Gemini

Prompt Techniques

System prompts, Few-Shot prompting, Chain-of-Thought (CoT) to improve quality and reasoning.

- System Prompt
- Few-Shot
- CoT / Reasoning

Structured Outputs

Use Instructor or Outlines to enforce schemas and get 100% reliable JSON extraction.

Instructor
Outlines

Token Management

Understand tokenization (tiktoken), calculate token costs and respect context window limits.

- tiktoken
- Cost calculation
- Context limits
- Max output tokens

BEST RESOURCES

COURSES

- DeepLearning.AI – Building Systems with the ChatGPT API
- Anthropic – Claude Prompt Engineering Course
- Google – Gemini API Quickstart

DOCUMENTATION

- OpenAI Developer Docs
- Anthropic Claude Docs
- Google Gemini API Docs
- tiktoken GitHub Docs
- Instructor Docs
- Outlines Docs

YOUTUBE CHANNELS

- Alex Volkov
- Prompt Engineering
- AI Explained
- Two Minute Papers (LLM concepts)

TOKEN COST EXAMPLE (gpt-4o)

TOKENS	INPUT COST	OUTPUT COST	EXAMPLE USE CASE
1K tokens	~$0.005	~$0.015	Short answers
10K tokens	~$0.05	~$0.15	Medium summaries
100K tokens	~$0.50	~$1.50	Long documents / RAG

Always monitor usage and set max_tokens to control spend.

PROMPTING CHEAT SHEET

- Be specific, clear and provide context.
- Define the output format explicitly.
- Use few-shot examples for complex tasks.
- Break down tasks & ask step-by-step (CoT).
- Validate with tools or structured outputs.

WHAT TO SKIP

Spending weeks memorizing 50+ obscure "prompt hacks".

Focus on fundamentals, patterns and real-world use cases.

REAL-WORLD EXAMPLES

- Extract Entities from Legal Contracts
- Parse Medical Reports into Structured JSON
- Product Data Normalization from Catalogs
- Support Ticket Summarization & Tagging

RESUME BULLET POINT YOU GET

> "Integrated OpenAI/Claude/Gemini APIs with Instructor for structured outputs, implemented token management and built reliable LLM-powered features in production."

Project 3:

AUTOMATED INVOICE & DOCUMENT PARSER

Turn messy documents into clean, structured data using LLMs and schema enforcement.



UNSTRUCTURED PDF INVOICE



Messy, inconsistent formats

VISION LLM / OCR

GPT-4o Vision

Claude 3.5

Gemini 1.5 Pro

Extract text & understand layout

INSTRUCTOR STRUCTURED SCHEMA

```
{
  "vendor": "Acme Corp",
  "invoice_id": "INV-10023",
  "date": "2025-05-15",
  "items": [
    { "name": "Widget",
    "qty": 2, "price": 199.99 },
    { "name": "Service Fee",
    "qty": 1, "price": 49.99 }
  ],
  "tax": 30.00,
  "total": 479.97
}
```

100% valid Pydantic JSON output

SQL DATABASE RECORD

vendor	Acme Corp
invoice_id	INV-10023
date	2025-05-15
total	479.97
tax	30.00
...	...

Saved to PostgreSQL with retries

PROJECT SPECIFICATIONS

1. **Ingest Messy PDFs**
   Accept uploaded invoice/receipt PDFs of any layout and quality.
2. **Extract with Vision LLM / OCR**
   Use GPT-4o Vision / Claude 3.5 / Gemini to read text, tables, and totals accurately.
3. **Structured Extraction with Instructor**
   Enforce a strict Pydantic schema for zero validation failures using Instructor/Outlines.
4. **Store in PostgreSQL**
   Insert structured records into Postgres with automated retry + idempotency.

TECH STACK

- Python 3.12
- OpenAI / Anthropic / Gemini APIs
- Instructor (Pydantic)
- PyPDF2 / pdfplumber
- PostgreSQL
- SQLAlchemy
- Tenacity (Retry Logic)
- FastAPI (API Interface)

KEY FEATURES

- Handles multi-page & noisy PDFs
- Line item table detection
- Tax, subtotal & total extraction
- 100% schema validation (Pydantic)
- Automatic retry on API failures
- Duplicate invoice detection
- REST API + Web UI (optional)
- Logs, monitoring & error alerts

SUCCESS METRICS

99.8%	Zero	<3s
Extraction Reliability	Pydantic Validation Failures	Avg. Processing Time per Document

EXAMPLE EXTRACTION OUTPUT

Sample Invoice (Input)



Structured JSON (Output)

```
{
  "vendor": "Acme Corp",
  "invoice_id": "INV-10023",
  "date": "2025-05-15",
  "items": [
    { "name": "Widget", "qty": 2,
      "price": 199.99, "amount": 399.98 },
    { "name": "Service Fee", "qty": 1,
      "price": 49.99, "amount": 49.99 }
  ],
  "tax": 30.00,
  "subtotal": 449.97,
} "total": 479.97
```

Database Record

vendor	Acme Corp
invoice_id	INV-10023
date	2025-05-15
subtotal	449.97
tax	30.00
total	479.97

WHAT YOU WILL LEARN

- Combining Vision LLMs + Structured Outputs
- Schema enforcement for reliable extraction
- Building production ETL-style pipelines
- Working with PostgreSQL & transactions
- Error handling, retries & observability

WHAT TO SKIP

- Building custom OCR from scratch
- Over-engineering UI/UX in early stages

RESUME BULLET POINT YOU GET

> "Engineered an automated document extraction pipeline parsing unstructured PDF invoices into strict SQL schemas with 99.8% extraction reliability."

Month 4: PRODUCTION RAG (BEYOND NAIVE DEMOS)

Build RAG systems that are accurate, reliable and ready for real-world production.



PDF UPLOAD
Ingest documents from any source

→

RECURSIVE CHUNKING
Smart chunking with 15% overlap

→

HYBRID SEARCH (DENSE + BM25)
Combine vector similarity with keyword search

→

COHERE RERANK
Cross-encoder rerank for top-k results

→

GROUNDED ANSWER
LLM generates answer with verified context

WHAT TO MASTER

Chunking Strategies
Recursive character chunking, semantic chunking, and 15% overlap.

Vector Databases
Qdrant, Pinecone, or pgvector setup, indexing & filtering.

Hybrid Search
Combine Dense vector similarity with Sparse BM25 keyword search.

Two-Stage Reranking
Use Cross-Encoder rerankers (Cohere Rerank v3 / BGE-Reranker).

TOOLS & TECHNOLOGIES

VECTOR DATABASES
Qdrant · Pinecone · pgvector

RERANKERS
cohere rerank-v3 · BGE Reranker · .jina Reranker

EMBEDDING MODELS
OpenAI text-embedding-3 · Cohere embed-english · BAAI bge-large-en

FRAMEWORKS
LangChain · LlamaIndex · Haystack

BEST PRACTICES

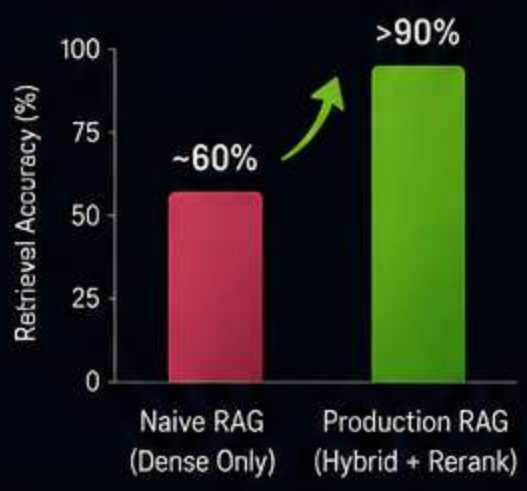
- Always chunk with overlap (10–15%)
- Store metadata (source, page, section, timestamp)
- Use metadata filtering for precise results
- Hybrid search > dense search alone
- Rerank top 20–50 results, not the entire set
- Log retrieval scores & monitor quality

TIP
Garbage in (document quality) = Garbage out (RAG answer).

NAIVE RAG vs PRODUCTION RAG

NAIVE RAG	PRODUCTION RAG
Fixed chunk size No overlap	Recursive chunking + 15% overlap
Only dense search	Hybrid search (Dense + BM25)
No reranking	Two-stage reranking
No metadata filtering	Netadata filtering
Inconsistent answers	Consistent, grounded answers
~60% Accuracy	>90% Accuracy

RETRIEVAL QUALITY IMPACT



REAL-WORLD USE CASES

- Enterprise Knowledge Bases
- Customer Support Assistants
- Legal / Compliance Research
- Healthcare Document Analysis
- Technical Documentation Q&A

FACT: Naive RAG fails ~40% of the time in production. Adding Hybrid Search + Reranking boosts retrieval accuracy **above 90%.**

> Good retrieval is the foundation of great generation.

Project 4:

ENTERPRISE KNOWLEDGE BASE WITH CITATIONS

Accurate answers. Trusted sources. Zero guessing.



1. USER QUESTION

What were the key financial highlights in Q1 FY24?

2. HYBRID RETRIEVAL (BM25 + DENSE)

Search across keywords (BM25) + semantic vectors (Qdrant)

3. RERANKER (CROSS-ENCODER)

Cohere Rerank v3 reorders top results by relevance

4. CONTEXT INJECTION TO LLM

Top-k passages injected as context with source metadata

5. UI WITH CITATIONS

Answer + clickable citations with exact page & line references

PROJECT SPECIFICATIONS

Ingest Enterprise Documents
Ingest 500+ PDFs (manuals, reports, filings). Support multi-page docs with tables, charts and footnotes.

Hybrid Search Implementation
Combine Dense Vector Search (Qdrant) with Sparse Keyword Search (BM25) using Reciprocal Rank Fusion (RRF).

Cross-Encoder Reranking
Use Cohere Rerank v3 / BGE-Reranker to re-rank top 50 results → return top 5-10 highly relevant chunks.

Citations & Source Highlighting
Show exact source (document, page, and paragraph/line). Enable clickable citations & preview pane for users.

KEY FEATURES

- Hybrid Retrieval (BM25 + Dense)
- Cross-Encoder Reranking
- Chunking with 15% overlap (Recursive)
- Metadata Filtering (by doc / date / section)
- Exact Citations with Page & Line numbers
- User-friendly UI with Search + Preview
- Export answers & citations to PDF/CSV
- Audit logs & query history

TECH STACK

Ingestion	LlamaParse / Unstructured
Chunking	Recursive Character + 15% Overlap
Embeddings	text-embedding-3-large (OpenAI) or bge-large-en
Vector DB	Qdrant
Keyword Search	ElasticSearch / OpenSearch (BM25)
Reranker	Cohere Rerank v3 (or BGE-Reranker)
LLM	GPT-4o / Claude 3.5 / Gemini 1.5
Backend	FastAPI
UI	Next.js / Streamlit

EXAMPLE: ANSWER WITH CITATIONS

Q: What were the key financial highlights in Q1 FY24?

ANSWER

In Q1 FY24, the company reported total revenue of $1.24B, up 18% YoY. Gross margin improved to 63.4% compared to 60.1% in Q1 FY23. Operating income was $216M, reflecting a 22% increase YoY...

SOURCES

1. Q1 FY24 Earnings Report P. 32, L. 14-18
2. Management Discussion P. 57, L. 9-12
3. Financial Statements P. 89, L. 3-7

Preview Source | Q1 FY24 Earnings Report.pdf | Page 32

View All Sources →

> Total revenue for the first quarter was $1.24 billion, an increase of 18% compared to $1.09 billion in the first quarter of 2023...

IMPACT & RESULTS (EXAMPLE)

85% Reduction in Hallucinations

90%+ Retrieval Accuracy (Top-5)

2.3s Avg. Response Time

> "Hybrid Search + Reranking = Accurate, Reliable, Trustworthy."

WHAT TO AVOID

- Only vector search (no keywords)
- No reranking (poor relevance)
- Large chunks (>1k tokens)
- No citation transparency

RESUME BULLET POINT YOU GET

> "Built enterprise RAG pipeline over 500+ corporate documents utilizing Hybrid Search (BM25 + Qdrant) and Cohere Reranking, cutting hallucinations by 85%."

Month 5: FUNCTION CALLING & MULTI-AGENT STATE MACHINES



Build agents that think, act, and deliver outcomes — reliably.

1. USER GOAL



I want sales insights for Q1 and latest market news.

2. SUPERVISOR ROUTER



Decides next best action & routes to the right tool

3A. TOOL: SQL QUERY (Database)



Query internal DB for sales metrics

3B. TOOL: WEB SEARCH (External)



Search the web for market & competitor updates

4. FINAL ACTION



Synthesize results & return actionable answer to user

WHAT TO MASTER

Tool / Function Calling

Let LLMs safely call external APIs, databases, and internal services using function schemas.

State Machines with LangGraph

Deterministic workflows with states, transitions, retries, and conditional routing for reliable agents.

Agent Memory

Short-term thread memory for context continuity. Long-term semantic memory for personalization and knowledge retention.

Human-in-the-Loop (HITL)

Require human approval for sensitive actions like sending emails, payments, or deleting records.

AGENT STATE MACHINE (LANGGRAPH)



MEMORY ARCHITECTURE

SHORT-TERM MEMORY

(Thread / Conversation)

- Recent messages
- Tool call results
- Current context window

LONG-TERM MEMORY

(User / Semantic)

- User preferences
- Past interactions
- Document embeddings
- Facts & entities

HUMAN-IN-THE-LOOP (HITL)

Gate critical actions

- Send email
- Delete record
- Process payment
- Change permissions

Pause → Review → Approve / Reject

COMMON TOOLS AGENTS CAN USE

SQL Database · REST APIs · Web Search · Python Functions · File Operations · Email / Notifs

BEST PRACTICES

Define clear goals & guardrails · Idempotent tool design · Timeouts & retries · Logging & observability · Cost & token monitoring · Evaluation & testing

WHAT TO SKIP

- Unconstrained infinite loops
- Autonomous swarms with no guardrails
- Agents that act without observability
- Over-engineered ultra-complex graphs

REAL-WORLD EXAMPLES

Data Analyst Agent · Research Assistant · Customer Support Agent · Internal Ops Automation · Compliance Checker

RESUME BULLET POINT YOU GET

> Designed and implemented multi-agent workflows with LangGraph, function calling, and human-in-the-loop safeguards to automate complex business tasks with 99%+ reliability.

Project 5:

AUTONOMOUS MARKET RESEARCH AGENT



An end-to-end autonomous agent that researches, analyzes, and delivers actionable market insights.

1. TOPIC INPUT
User provides research topic or goal

2. SEARCH NODE
Search relevant info using Tavily API (web + news + blogs)

TAVILY

3. SCRAPER NODE
Extract article content, metadata & dates from top results

4. ANALYST NODE
Summarize key trends, compare insights, identify opportunities

5. HITL REVIEW GATE
Human reviews & approves before finalizing report

6. PDF REPORT
Generate clean, branded PDF report with references

PROJECT SPECIFICATIONS

1 Web Search
Use Tavily API to search the web for high-quality, recent sources.

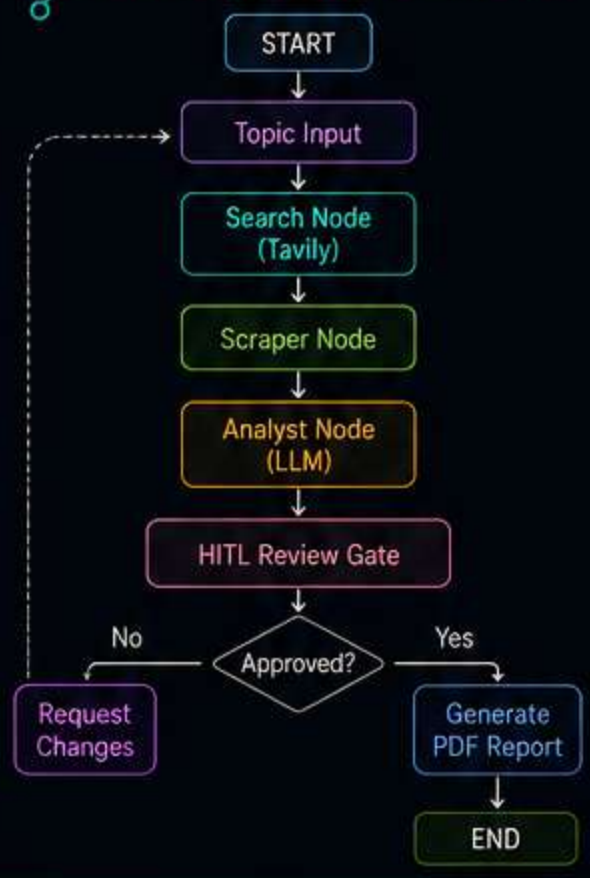
2 Content Scraping
Scrape full article content, extract metadata (title, source, date, url).

3 Analysis & Synthesis
LLM analyzes, clusters insights, identifies trends, risks & opportunities.

4 Human-in-the-Loop
Human reviews summary & approves or requests changes.

5 Report Generation
Generate final structured report & export as PDF with citations and source links.

LANGGRAPH STATE MACHINE



TECH STACK

- **LangGraph** — State machine orchestration
- **Tavily API** — Web search
- **BeautifulSoup / Scrapy** — Web scraping
- **OpenAI / Claude** — Analysis & summarization
- **Pydantic** — Structured outputs & schemas
- **WeasyPrint / ReportLab** — PDF generation
- **PostgreSQL + pgvector** — Memory & long-term storage
- **Streamlit / Next.js** — Dashboard & UI

KEY FEATURES

- Multi-step autonomous workflow
- Dynamic tool calling (search, scrape, analyze)
- Short-term thread memory with LangGraph
- Human-in-the-Loop approval gate
- Source citations in final report
- Export to PDF with branding & TOC

SAMPLE OUTPUT (REPORT PREVIEW)



TOP MARKET TRENDS
- AI adoption in SMBs is accelerating...
- Rising demand for automation...
- Shift towards vertical AI solutions...

KEY OPPORTUNITIES
- Niche AI copilots for finance...
- Customer support automation...
- Data privacy compliant AI tools...

SOURCES (Click to view)
1. Forbes – AI in SMBs (May 2024)
2. McKinsey – The State of AI (2024)
3. Gartner – Top Tech Trends (2024)

SUCCESS METRICS

90%+ Source Relevance

85%+ Summary Accuracy (Human Eval)

5–10 min End-to-End Research Time

WHAT TO AVOID

- Infinite loops & runaway agents
- No human oversight
- Unverified or low-quality sources
- Over-complicated agent graphs

RESUME BULLET POINT YOU GET

> "Architected a multi-step autonomous research agent with LangGraph, incorporating dynamic tool calling, memory management, and Human-in-the-Loop gates."

Month 6: LLM EVALUATION, GUARDRAILS & SERVING



Ship reliable, safe and measurable LLM systems that you can trust in production.

1. USER PROMPT
User asks a question or provides input

2. LLAMA GUARD 3
Detect & block harmful content, jailbreaks, and policy violations

3. vLLM GPU ENGINE
High-throughput inference with vLLM (Llama-3 / Mistral)

4. PRESIDIO PII MASKING
Detect and mask PII in prompts & responses (email, phone, SSN, etc.)

5. RAGAS CI/CD EVAL
Evaluate quality on every commit: correctness, groundedness, faithfulness

1. EVALUATION FRAMEWORKS

Measure what matters.
Catch regressions early.

RAGAS | TruLens

Key Metrics

- Context Relevance (Are retrieved docs relevant?)
- Faithfulness (Is the answer supported by context?)
- Answer Relevancy (Does it answer the question?)
- Correctness (Is the answer factually correct?)
- Groundedness (No hallucinations)

Run evaluations on datasets & real user queries.

2. INGRESS / EGRESS GUARDRAILS

Protect users, data, and your application.

Ingress (Input) Guardrails

- Llama Guard 3 — Safety classification
- NeMo Guardrails — Policy enforcement
- Prompt Injection & Jailbreak detection

Egress (Output) Guardrails

- Microsoft Presidio — PII detection & masking
- Toxicity & Hate content filtering
- Content Moderation custom policies

Defense in depth = Safe inputs, safe outputs.

3. LOCAL MODEL SERVING

Serve open-source models at scale and efficiency.

vLLM | Ollama

- Llama-3 / Mistral / Mixtral Qwen and more
- vLLM: High throughput, low latency, OpenAI API compatible
- Pagination / Streaming / Function calling
- Quantization (AWQ, GPTQ) for cost efficiency
- GPU Optimization Tensor parallelism, KV cache

OpenAI compatible API Drop-in replacement.

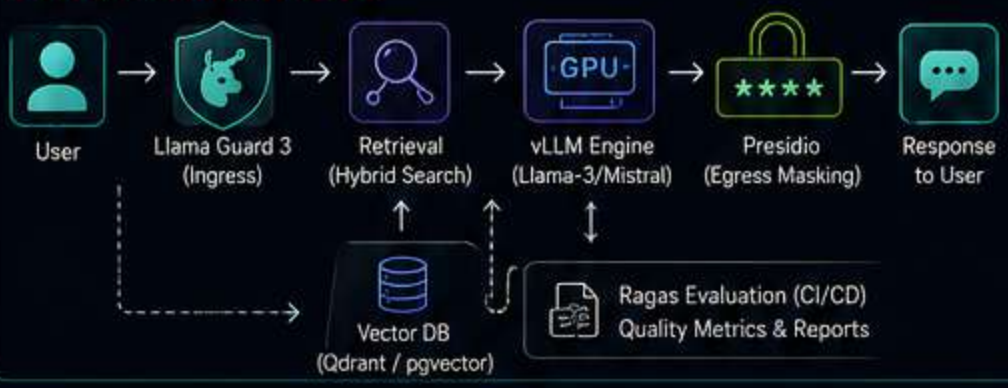
4. CI/CD AUTOMATION

Make quality a gate, not a guess.

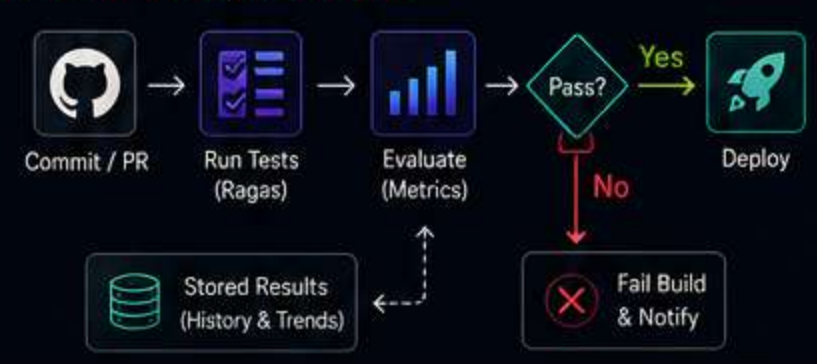
- GitHub Actions / GitLab CI — Automate evaluation pipeline
- Ragas / TruLens Tests — Run on every commit / PR
- Regression Detection — Fail build on quality drop
- Datasets — Golden set + Real user conversations
- Artifacts & Reports — Store results, trends & dashboards

No deploy without passing quality gates.

REFERENCE ARCHITECTURE



EVALUATION & CI/CD PIPELINE



BEST PRACTICES

- Version prompts, models & datasets
- Monitor latency, cost, and quality
- Alert on guardrail violations
- Red-team your system regularly
- Human-in-the-loop for high-risk actions

WHAT TO SKIP

- Skipping evaluations
- Running models, without guardrails
- Hardcoding secrets in code
- Ignoring PII & compliance
- Deploying without monitoring

PRODUCTION CHECKLIST

- Guardrails enabled (input + output)
- Evaluation suite passing in CI/CD
- Observability (logs, traces, metrics)
- Cost & latency within thresholds
- Rollback strategy in place

FACT

Organizations with LLM evals & guardrails in CI/CD reduce production incidents by **70%+.**

RESUME BULLET POINT YOU GET

> "Architected a multi-step autonomous research agent with LangGraph, incorporating dynamic tool calling, memory management, and Human-in-the-Loop gates."

Resume Strategy: Projects That Get Interviews

Hiring managers look for **impact, complexity,** and **execution** — not just model accuracy.

Focus on depth, real-world impact, and production readiness — not toy projects.

DELETE IMMEDIATELY

RED FLAGS



Titanic Survival Prediction

- Overdone & outdated
- Teaches basic ML only
- No real-world complexity
- Won't stand out



Iris Flower Classification

- Extremely trivial dataset
- Solved in 10 minutes
- No engineering depth
- Not interview impressive



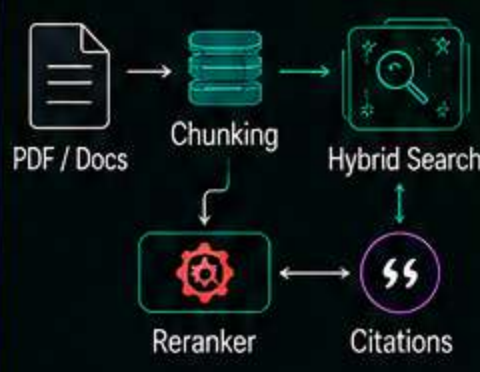
Generic 10-line ChatGPT Wrapper

- No real problem solved
- No architecture or depth
- Everyone has a clone
- Zero differentiation

Avoid projects that are basic, generic, or lack engineering depth.

KEEP & HIGHLIGHT

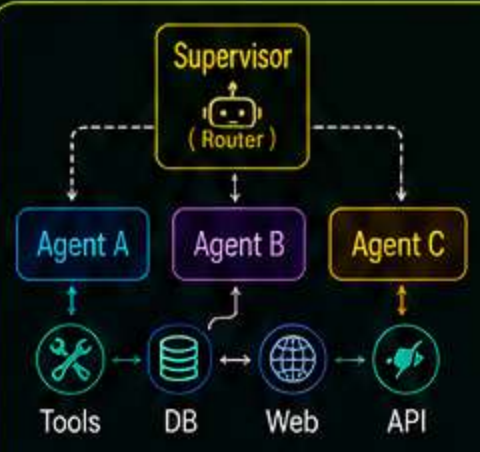
HIRING GOLD



Enterprise Hybrid RAG with Citations & Reranking

- Solves real enterprise need
- Hybrid Search + Reranking
- Citations = trust & reliability
- High impact & interview gold

Impact: High



LangGraph Multi-Agent System with Tool Calling

- Demonstrates orchestration
- Tool / Function calling
- State machines & memory
- Complexity that gets noticed

Impact: Very High



Containerized LLM API with Guardrails & Eval

- Production-ready mindset
- PII masking & safety
- Automated evaluation (CI/CD)
- Shows engineering maturity

Impact: Very High

Build projects that solve real problems with production-grade execution.

QUICK CHECKLIST

- Does it solve a real problem?
- Is it technically challenging?
- Did you build it end-to-end?
- Does it use modern tools?
- Can you deploy & demo it?
- Can you explain the trade-offs?

PRO TIP

Quality over quantity.
3 strong projects > 10 weak projects.

Show depth.
Show ownership.
Show results.



REMEMBER Hiring managers don't care how many models you trained. They care about the **impact you created.**

How to Pass AI Engineering Interviews: The 3 Rounds



Skills. System Design. Execution. That's how you get offers.

1 ROUND 1 — Python & Data Engineering Screening

45 min

Goal: Test your fundamentals and real-world backend skills.

WHAT TO EXPECT

- **Async Programming** — asyncio, tasks, concurrency
- **Pydantic Schemas** — Data validation, nested models
- **API Architecture** — FastAPI, REST design, auth
- **SQL Queries** — Joins, CTEs, aggregations

TIP: Code cleanly. Think out loud. Handle edge cases.

2 ROUND 2 — AI / LLM System Design

60 min

Goal: Design scalable, reliable AI systems end-to-end.

WHAT TO EXPECT

- **Requirements & Constraints** — Use case, scale, volume, latency
- → **RAG Pipeline Design** — Chunking strategy, embedding model
- → **Vector DB at Scale** — Qdrant / Pinecone, sharding, metadata
- → **Latency & SLAs** — Target latency, throughput, cost
- → **Guardrails & Safety** — PII, hallucination, content filters

TIP: Clarify assumptions, draw diagrams, discuss trade-offs.

3 ROUND 3 — Live Coding & Architecture Defense

60 min

Goal: Implement solutions and defend your design choices.

WHAT TO EXPECT

- **Live Coding** — Implement a feature or tool-calling loop
- → **Debugging & Troubleshooting** — Fix retrieval failures, errors, edge cases
- → **Architecture Defense** — Why this design? Alternatives? Trade-offs?
- → **Communication Matters** — Explain clearly. Show ownership. Stay calm.

TIP: Write working code. Reason step-by-step. Own your decisions.

INTERVIEW SUCCESS FORMULA

- **Strong Fundamentals** — Python, SQL, APIs, Data Structures
- **System Thinking** — Design scalable, reliable AI systems
- **Execution & Communication** — Code, debug, explain, iterate

TOPICS TO REVIEW

- Python (asyncio, generators, decorators)
- Pydantic, FastAPI, SQL
- LangChain, RAG, Chunking
- Vector DBs (Qdrant / Pinecone / pgvector)
- LLM Evaluation (Ragas, Metrics)
- Agent Frameworks (LangGraph)
- Deployment, Observability, Guardrails

REMEMBER

> Companies don't just hire who can code. They hire who can **build, think, and ship** AI systems that work in the real world.

PREPARE. PRACTICE. BUILD.

Your portfolio gets you the interview. Your skills get you the offer. 🚀

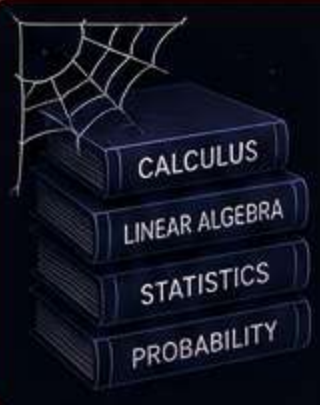
resnur.ai

COMMON MISTAKES

5 Traps Keeping You Stuck in 'Tutorial Purgatory'



Avoid these traps. Follow the fixes. **Ship real-world systems.**



TRAP 1

Spending months learning calculus before writing code

Analysis paralysis. You never ship anything real.

FIX 1

Learn top-down by building working systems.

Build first. Understand deeply along the way.





TRAP 2

Writing messy code in Jupyter Notebooks only

Hard to maintain, test, collaborate or deploy.

FIX 2

Build modular .py packages, APIs & Dockerfiles.

Clean structure. Easy to test, reuse and deploy.

```
my.project/
├─ app/
├─ models/
├─ services/
├─ utils/
├─ main.py
├─ requirements.txt
└─ Dockerfile
```



TRAP 3

Over-relying on high-level framework magic

You don't know what's happening under the hood.

FIX 3

Know raw API calls and embedding math under the hood.

Understanding = Power. Design better. Debug faster.

```
# Raw API Call
response = requests.post(
    url, headers=headers,
    json=payload
)

# Embedding Similarity
similarity = np.dot(a, b) /
    np.linalg.norm(a) *
    np.linalg.norm(b)
)
```



TRAP 4

Never measuring accuracy

You ship systems with no idea if they actually work.

FIX 4

Add Ragas evaluation metrics to your projects.

- Context Relevance
- Faithfulness
- Answer Relevancy
- Groundedness





TRAP 5

Not deploying projects live

No real users. No feedback. No impact.

FIX 5

Host your APIs on cloud (AWS/GCP/Modal/Render) with live URLs.

Real users. Real feedback. Real growth.



REMEMBER Progress comes from shipping, not watching.
Build → Measure → Ship → Improve → Repeat.

BUILD. MEASURE. SHIP.
THAT'S HOW YOU WIN.

SUMMARY

Your 6-Month Roadmap to Landing an AI Role



🎯 A clear plan. Real projects. Production skills. Interview ready.

Follow this roadmap and stay consistent.
6 months from now, you won't be a beginner—you'll be a production-ready AI Engineer companies fight to hire.

MONTH 1



Python, Pydantic, FastAPI & Docker

- Advanced Python (typing, dataclasses, async)
- Pydantic for robust data validation
- Build REST APIs with FastAPI
- Containerize with Docker & Docker Compose

Outcome: Ship a production-ready API service in Docker.

MONTH 2



SQL, Vector Math & Semantic Search

- Advanced SQL (joins, CTEs, window functions)
- Embeddings & cosine similarity
- Build semantic search with Qdrant / Pinecone
- Hybrid search (BM25 + Vector) basics

Outcome: Build a semantic search engine over real data.

MONTH 3



LLM SDKs & Structured JSON Outputs

- OpenAI / Azure OpenAI SDKs
- Function calling & JSON mode
- Pydantic models for structured outputs
- Error handling, retries & rate limiting

Outcome: Build reliable LLM apps with structured outputs.

MONTH 4

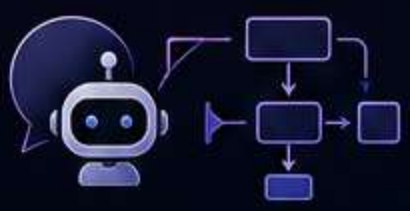


Production Hybrid RAG & Reranking

- Ingest & chunk large documents
- Hybrid search (BM25 + Vector)
- Rerank with Cross-Encoder / Cohere Rerank
- Citations with page & line level references

Outcome: Build enterprise RAG with citations that reduces hallucinations.

MONTH 5



AI Agents & LangGraph State Machines

- Tool / function calling (APIs, DBs, Search)
- LangGraph for agent orchestration
- Memory (short-term + long-term)
- Human-in-the-Loop (HITL) & approvals

Outcome: Build multi-step autonomous agents that get real work done safely.

MONTH 6



Evaluation, Guardrails & Live Deployment

- Evaluate with Ragas / TruLens
- Guardrails (Llama Guard 3, NeMo, Presidio)
- Serve models with vLLM / Ollama
- Deploy APIs (AWS/GCP/Modal/Render)
- CI/CD with automated eval on every commit

Outcome: Ship, monitor & scale production AI systems with confidence.

📌 **SAVE** THIS POST for your 6-month study plan!

♻ **REPOST** to help freshers and career switchers break into AI!

FOLLOW RESNUR AI for daily production AI engineering breakdowns.